9장 우선순위 큐

우선순위 큐

- 우선순위 큐(**priority queue**): 우선순위를 가진 항목들을 저장하는 큐
- **FIFO** 순서가 아니라 우선 순위가 높은 데이터가 먼저 나가게 된다.





© 생능출판사 2019



우선순위 큐

- 가장 일반적인 큐: 스택이나 FIFO 큐를 우선순위 큐로 구현할 수 있다.

자료구조	삭제되는 요소
스택	가장 최근에 들어온 데이터
큐	가장 먼저 들어온 데이터
우선순위큐	가장 우선순위가 높은 데이터

- 응용분야:
  - 시뮬레이션 시스템(여기서의 우선 순위는 대개 사건의 시각이다.)
  - 네트워크 트래픽 제어
  - 운영 체제에서의 작업 스케쥴링



© 생능출판사 2019



우선순위큐 ADT

·객체: n개의 element형의 우선 순위를 가진 요소들의 모임

·연산:

- create() ::= 우선 순위큐를 생성한다.
- init(q) ::= 우선 순위큐 q를 초기화한다.
- is_empty(q) ::= 우선 순위큐 q가 비어있는지를 검사한다.
- is_full(q) ::= 우선 순위큐 q가 가득 찼는가를 검사한다.
- insert(q, x) ::= 우선 순위큐 q에 요소 x를 추가한다.
- delete(q) ::= 우선 순위큐로부터 가장 우선순위가 높은 요소를 삭제하고 이 요소를 반환한다.
- find(q) ::= 우선 순위가 가장 높은 요소를 반환한다.



© 생능출판사 2019



우선순위큐 ADT

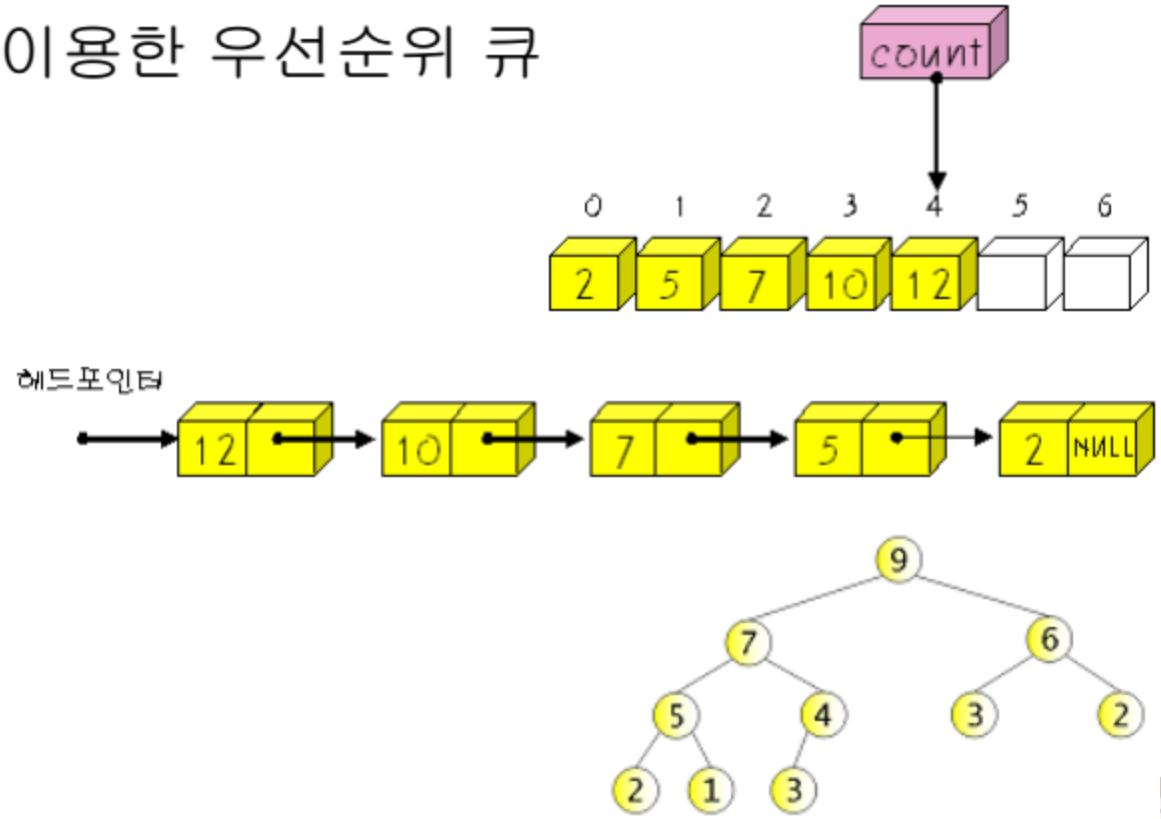
- 가장 중요한 연산은 insert 연산(요소 삽입), delete 연산(요소 삭제)이다.
- 우선순위 큐는 2가지로 구분
  - 최소 우선순위 큐
  - 최대 우선순위 큐



© 생능출판사 2019

우선순위 큐 구현방법

- 배열을 이용한 우선순위 큐
- 연결리스트를 이용한 우선순위 큐
- 히프(heap)를 이용한 우선순위 큐



© 생능출판사 2019



우선순위큐 구현방법

표현 방법	삽 입	삭 제
순서없는 배열	O(1)	O(n)
순서없는 연결 리스트	O(1)	O(n)
정렬된 배열	O(n)	O(1)
정렬된 연결 리스트	O(n)	O(1)
히프	O(log n)	O(log n)

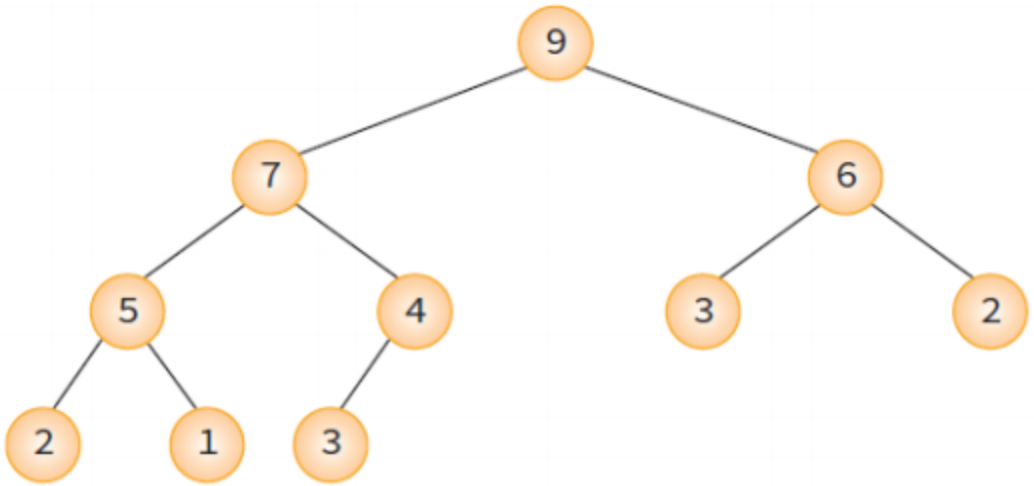


© 생능출판사 2019



히프(heap)란?

- 노드의 키들이 다음 식을 만족하는 완전이진트리
- *key(부모노드) ≥ key(자식노드)*





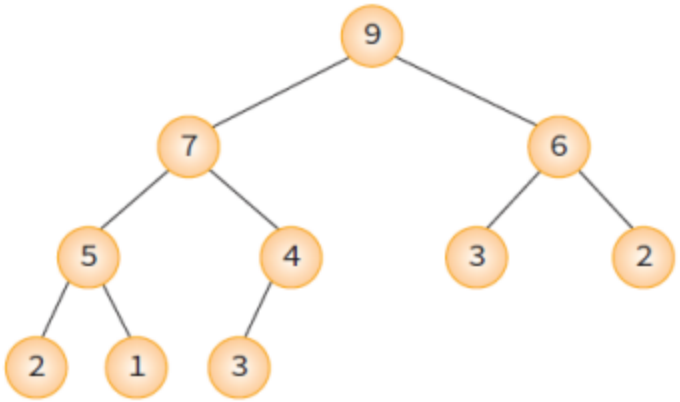
© 생능출판사 2019



히프의 종류

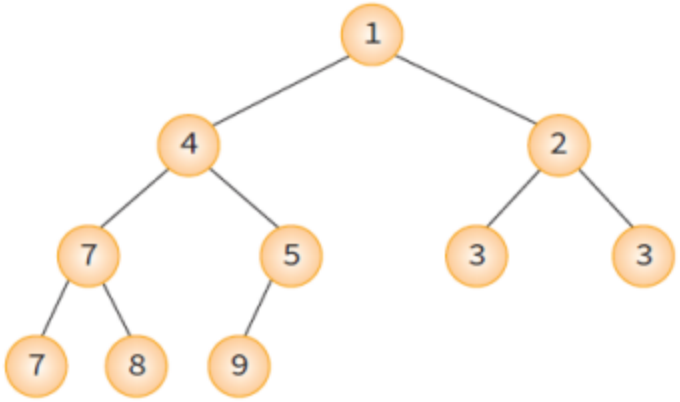
최대 히프(max heap):
부모 노드의 키값이 자식 노드의 키값보다 크거나 같은 완전 이진 트리

key(부모 노드) $\geq$ key(자식 노드)



최소 히프(min heap):
부모 노드의 키값이 자식 노드의 키값보다 작거나 같은 완전 이진 트리

key(부모 노드) $\leq$ key(자식 노드)



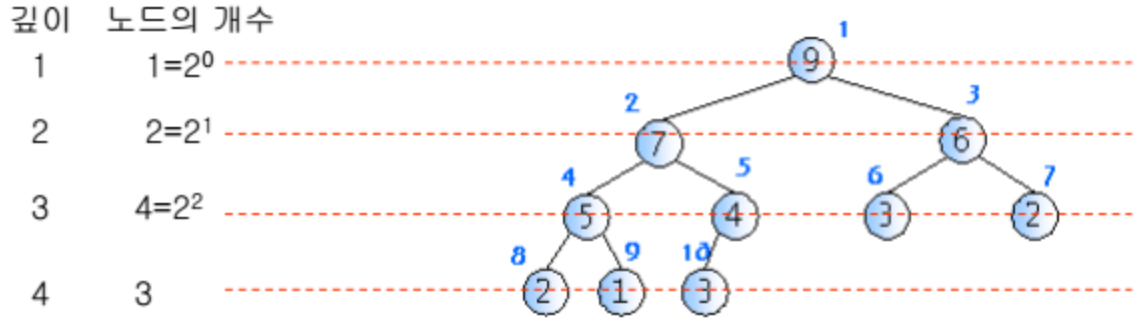


© 생능출판사 2019



히프의 높이

- n개의 노드를 가지고 있는 히프의 높이는 $\lceil \log_2(n+1) \rceil$
  - 히프는 완전이진트리
  - 마지막 레벨 h을 제외하고는 각 레벨 i에 2^{i-1}개의 노드 존재



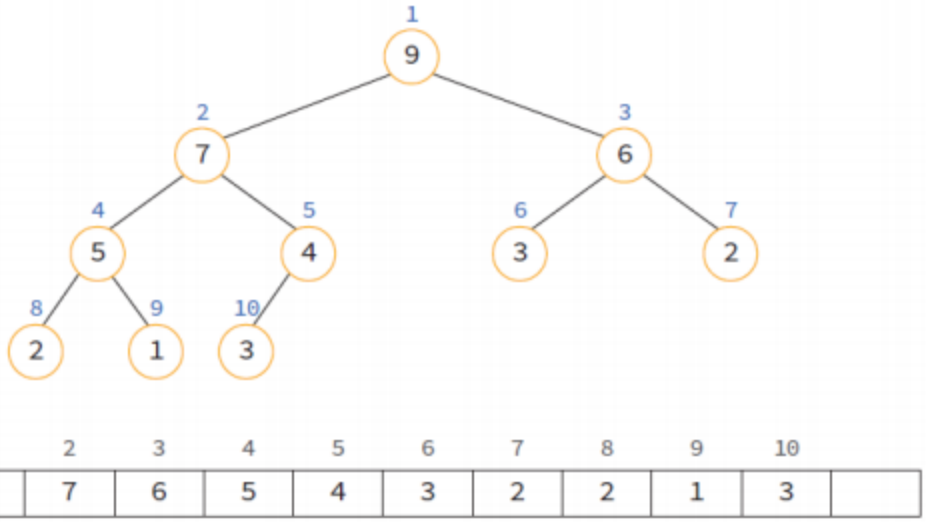


© 생능출판사 2019



히프의 구현방법

- 히프는 배열을 이용하여 구현
  - 완전이진트리이므로 각 노드에 번호를 붙일 수 있다
  - 이 번호를 배열의 인덱스라고 생각



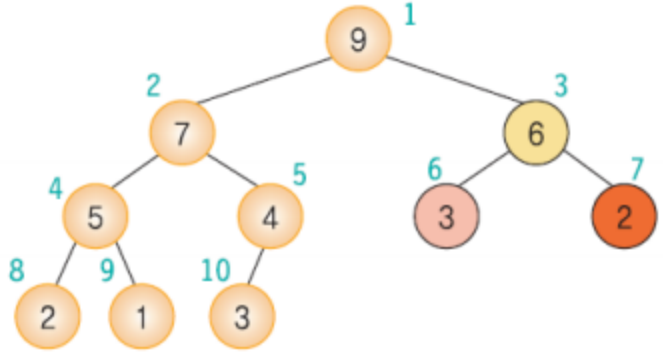


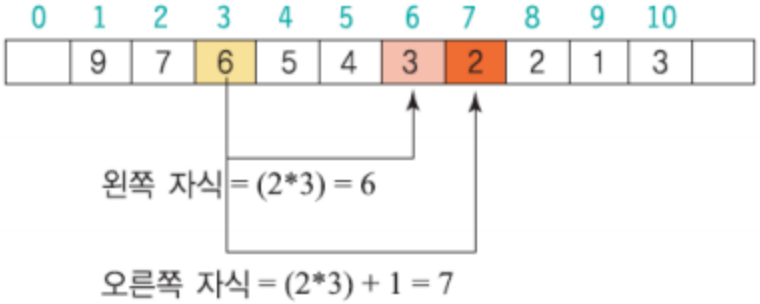
© 생능출판사 2019



히프의 구현방법

- 부모노드와 자식노드를 찾기가 쉽다.
  - 왼쪽 자식의 인덱스 = (부모의 인덱스)*2
  - 오른쪽 자식의 인덱스 = (부모의 인덱스)*2 + 1
  - 부모의 인덱스 = (자식의 인덱스)/2









히프의 정의

```
#define MAX_ELEMENT 200
typedef struct {
        int key;
} element;

typedef struct {
        element heap[MAX_ELEMENT];
        int heap_size;
} HeapType;

HeapType heap;                  // 정적 메모리 할당 사용
HeapType *heap = create();      // 동적 메모리 할당 사용
```



© 생능출판사 2019

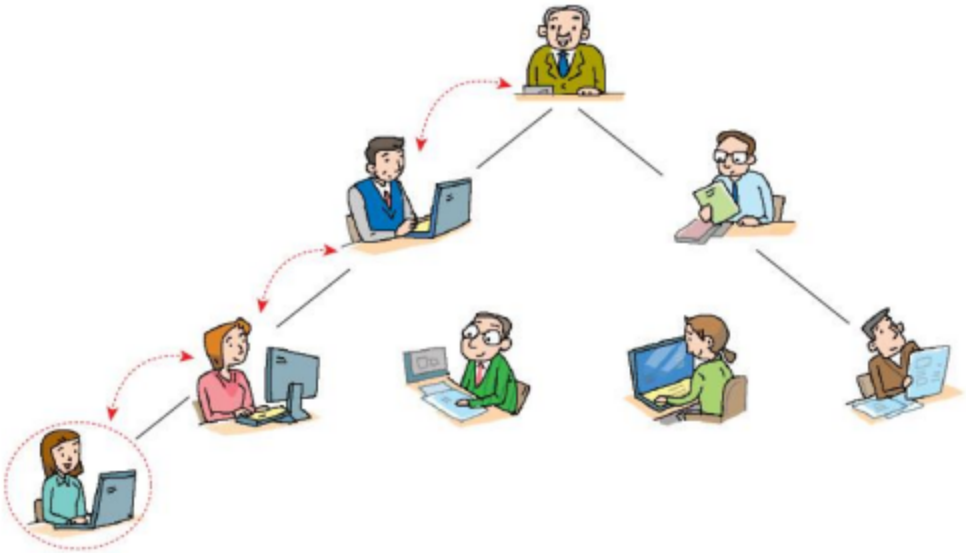


히프에서의 삽입

- 히프에 있어서 삽입 연산은 회사에서 신입 사원이 들어오면 일단 말단 위치에 앉힌 다음에, 신입 사원의 능력을 봐서 위로 승진시키는 것과 비슷

(1) 히프에 새로운 요소가 들어 오면, 일단 새로운 노드를 히프의 마지막 노드에 이어서 삽입

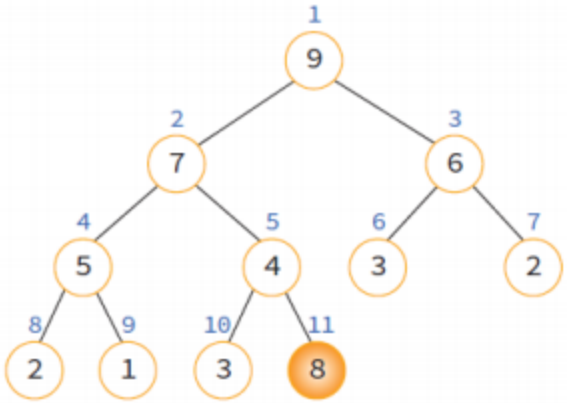
(2) 삽입 후에 새로운 노드를 부모 노드들과 교환해서 히프의 성질을 만족

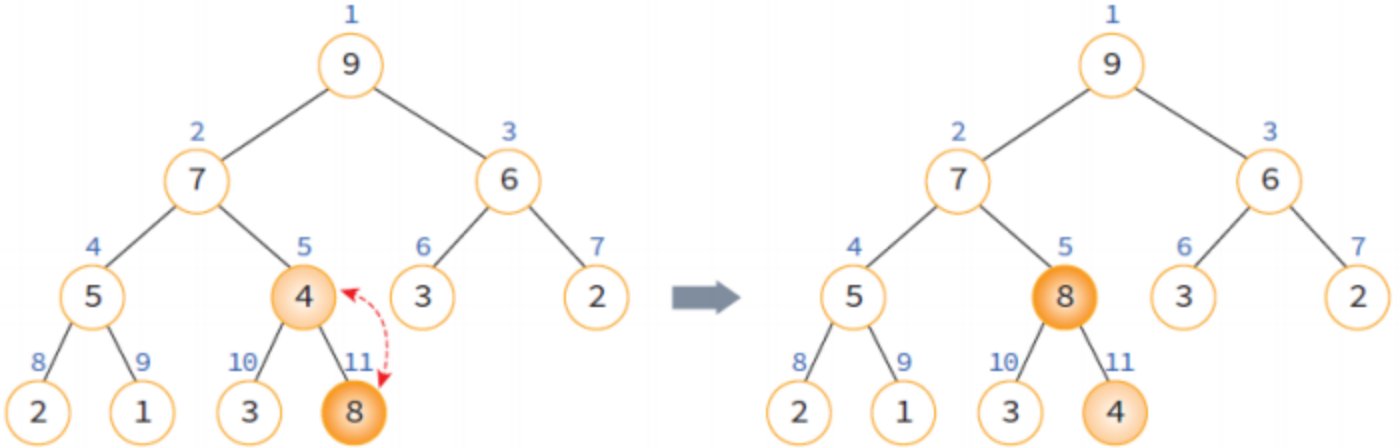




© 생능출판사 2019

upheap 연산



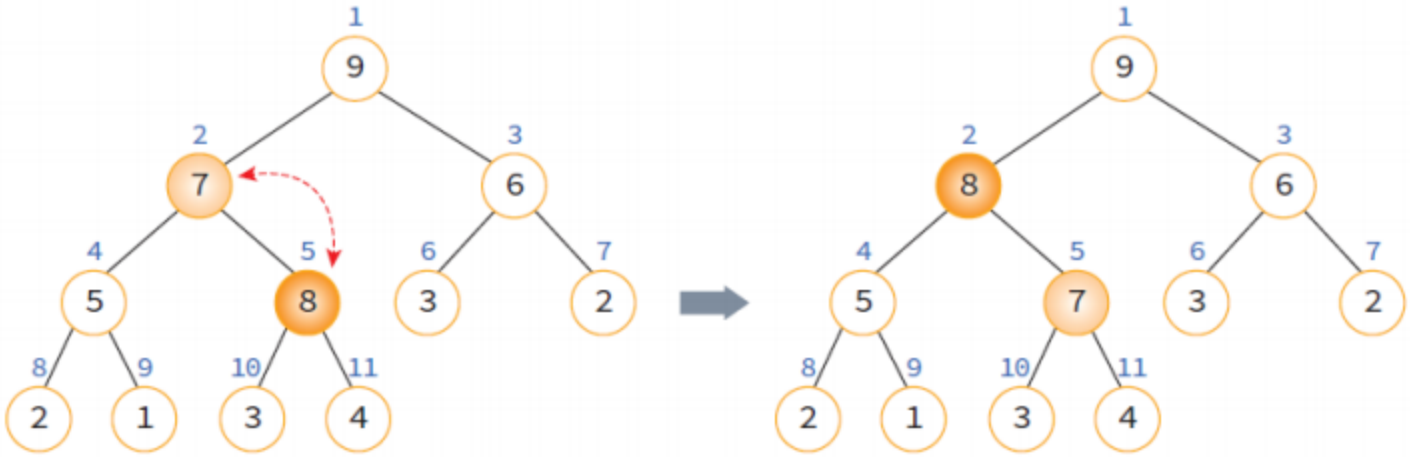




© 생능출판사 2019



upheap 연산





© 생능출판사 2019



upheap 알고리즘

```
insert_max_heap(A, key)


 heap_size ← heap_size + 1;
 i ← heap_size;
 A[i] ← key;
 while i ≠ 1 and A[i] > A[PARENT(i)] do
                          A[i] ↔ A[PARENT(i)];
                          i ← PARENT(i);
```



© 생능출판사 2019



삽입 프로그램

```
// 현재 요소의 개수가 heap_size인 히프 h에 item을 삽입한다.
// 삽입 함수
void insert_max_heap(HeapType *h, element item)
{
   int i;
   i = ++(h->heap_size);

   // 트리를 거슬러 올라가면서 부모 노드와 비교하는 과정
   while((i != 1) && (item.key > h->heap[i/2].key)) {
            h->heap[i] = h->heap[i/2];
            i /= 2;
    }
   h->heap[i] = item;     // 새로운 노드를 삽입
}
```



© 생능출판사 2019



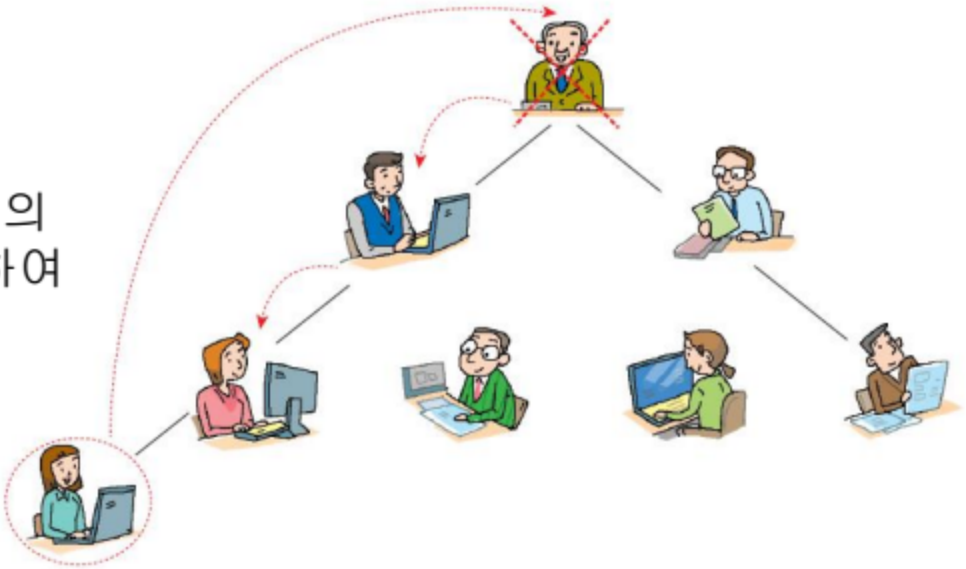
히프에서의 삭제

- 최대 히프에서의 삭제는 가장 큰 키값을 가진 노드를 삭제하는 것을 의미

-> 따라서 루트 노드가 삭제된다.

- 삭제 연산은 회사에서 사장의 자리가 비게 되면 먼저 제일 말단 사원을 사장 자리로 올린 다음에, 능력에 따라 강등시키는 것과 비슷하다.

(1) 루트 노드를 삭제한다
(2) 마지막 노드를 루트 노드로 이동한다.
(3) 루트에서부터 단말 노드까지의 경로에 있는 노드들을 교환하여 히프 성질을 만족시킨다.

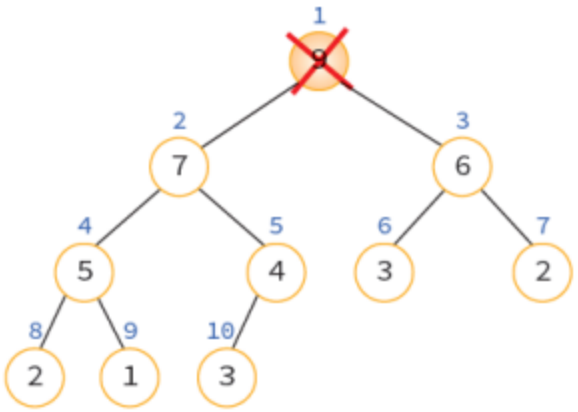


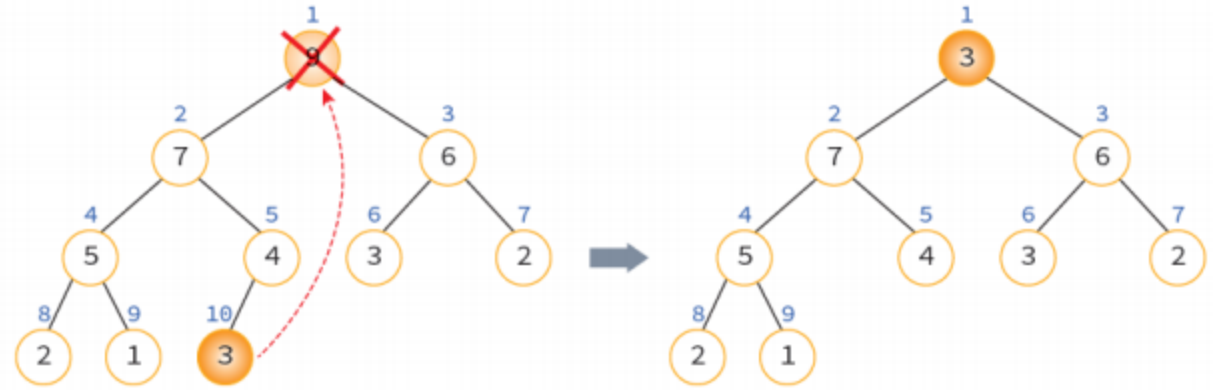


© 생능출판사 2019



downheap 알고리즘



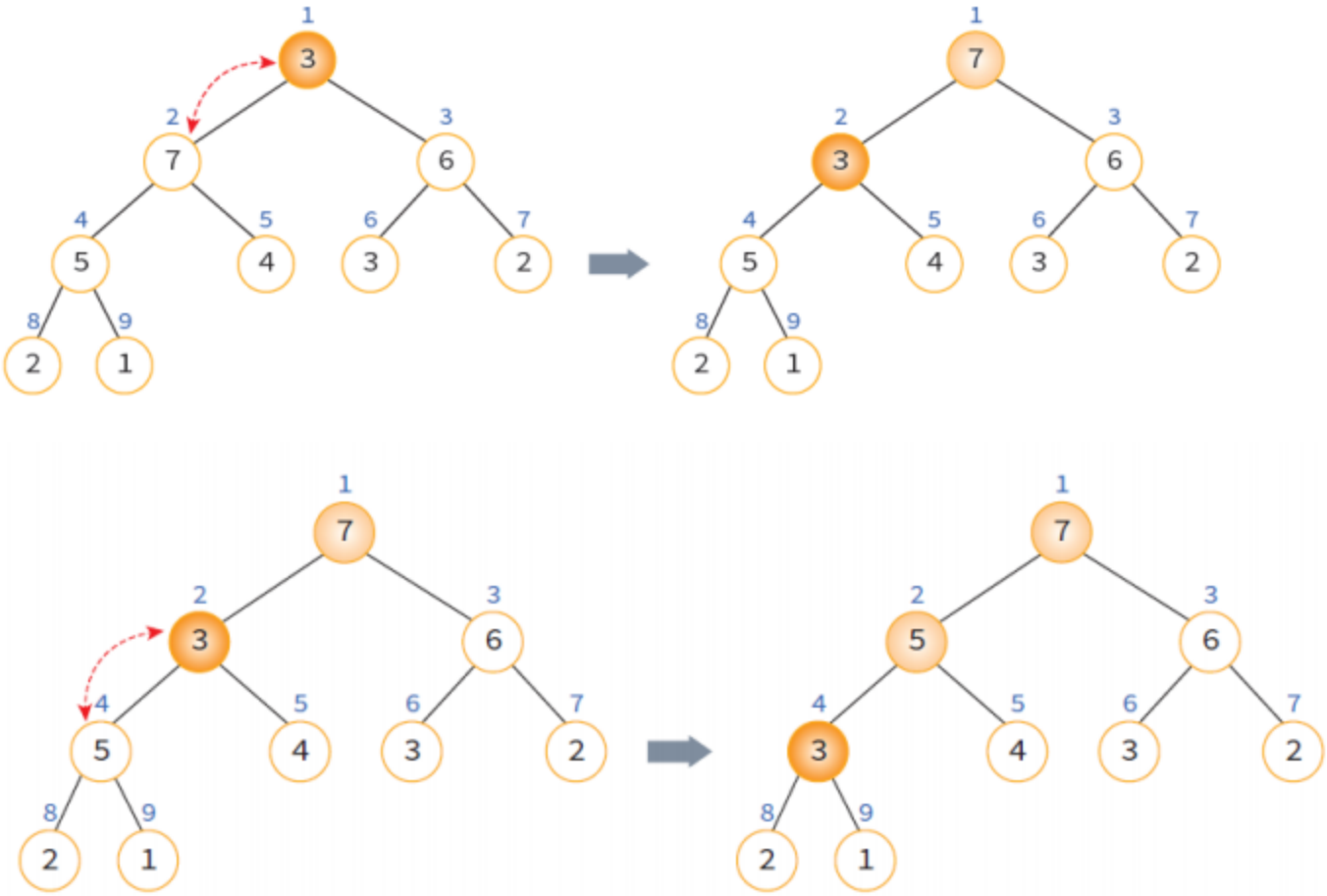




© 생능출판사 2019



downheap 알고리즘





© 생능출판사 2019



downheap 알고리즘

```
delete_max_heap(A):

item ← A[1];
A[1] ← A[heap_size];
heap_size←heap_size-1;
i ← 2;
while i ≤ heap_size do
        if i < heap_size and A[i+1] > A[i]
                then largest ← i+1;
                else largest ← i;
        if A[PARENT(largest)] > A[largest]
                then break;
        A[PARENT(largest)] ↔ A[largest];
        i ← CHILD(largest);

return item;
```



© 생능출판사 2019



삭제 프로그램

```
// 삭제 함수
element delete_max_heap(HeapType *h)
{
    int parent, child;
    element item, temp;

    item = h->heap[1];
    temp = h->heap[(h->heap_size)--];
    parent = 1;
    child = 2;
    while( child <= h->heap_size ) {
                // 현재 노드의 자식노드중 더 큰 자식노드를 찾는다.
                if( ( child < h->heap_size ) &&
                    (h->heap[child].key) < h->heap[child+1].key)
                    child++;
                if( temp.key >= h->heap[child].key ) break;
                // 한단계 아래로 이동
                h->heap[parent] = h->heap[child];
                parent = child;
                child *= 2;
    }
    h->heap[parent] = temp;
    return item;
 }
```



© 생능출판사 2019



전체 프로그램

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_ELEMENT 200
typedef struct {
        int key;
} element;
typedef struct {
        element heap[MAX_ELEMENT];
        int heap_size;
} HeapType;


// 생성 함수
HeapType* create()
{
        return (HeapType*)malloc(sizeof(HeapType));
}
// 초기화 함수
void init(HeapType* h)
{
        h->heap_size = 0;
}
```



© 생능출판사 2019



전체 프로그램

```
// 현재 요소의 개수가 heap_size인 히프 h에 item을 삽입한다.
// 삽입 함수
void insert_max_heap(HeapType* h, element item)
{
        int i;
        i = ++(h->heap_size);

        //  트리를 거슬러 올라가면서 부모 노드와 비교하는 과정
        while ((i != 1) && (item.key > h->heap[i / 2].key)) {
                h->heap[i] = h->heap[i / 2];
                i /= 2;
        }
        h->heap[i] = item;     // 새로운 노드를 삽입
}
```



© 생능출판사 2019



전체 프로그램

```
// 삭제 함수
element delete_max_heap(HeapType* h)
{
        int parent, child;
        element item, temp;

        item = h->heap[1];
        temp = h->heap[(h->heap_size)--];
        parent = 1;
        child = 2;
        while (child <= h->heap_size) {
                // 현재 노드의 자식노드 중 더 작은 자식노드를 찾는다.
                if ((child < h->heap_size) &&
                    (h->heap[child].key) < h->heap[child + 1].key)
                        child++;
                if (temp.key >= h->heap[child].key) break;
                // 한 단계 아래로 이동
                h->heap[parent] = h->heap[child];
                parent = child;
                child *= 2;
        }
        h->heap[parent] = temp;
        return item;
}
```



© 생능출판사 2019



전체 프로그램

```
int main(void)
{
        element e1 = { 10 }, e2 = { 5 }, e3 = { 30 };
        element e4, e5, e6;
        HeapType* heap;

        heap = create();        // 히프 생성
        init(heap);             // 초기화

                                        // 삽입
        insert_max_heap(heap, e1);
        insert_max_heap(heap, e2);
        insert_max_heap(heap, e3);

        // 삭제
        e4 = delete_max_heap(heap);
        printf("< %d > ", e4.key);
        e5 = delete_max_heap(heap);
        printf("< %d > ", e5.key);
        e6 = delete_max_heap(heap);
        printf("< %d > \n", e6.key);

        free(heap);
        return 0;
}
```





실행결과

```
< 30 > < 10 > < 5 >
```



© 생능출판사 2019



히프의 복잡도 분석

- 삽입 연산에서 최악의 경우, 루트 노드까지 올라가야 하므로 트리의 높이에 해당하는 비교 연산 및 이동 연산이 필요하다. ->O(log n)

- 삭제도 최악의 경우, 가장 아래 레벨까지 내려가야 하므로 역시 트리의 높이 만큼의 시간이 걸린다. ->O(log n)



© 생능출판사 2019



히프 정렬

- 히프를 이용하면 정렬 가능
- 먼저 정렬해야 할 n개의 요소들을 최대 히프에 삽입
- 한번에 하나씩 요소를 히프에서 삭제하여 저장하면 된다.
- 삭제되는 요소들은 값이 증가되는 순서(최소히프의 경우)
- 하나의 요소를 히프에 삽입하거나 삭제할 때 시간이 O(log n) 만큼 소요되고 요소의 개수가 n개이므로 전체적으로 O(n log n)시간이 걸린다. (빠른편)
- 히프 정렬이 최대로 유용한 경우는 전체 자료를 정렬하는 것이 아니라 가장 큰 값 몇 개만 필요할 때이다.
- 이렇게 히프를 사용하는 정렬 알고리즘을 히프 정렬이라고 한다.

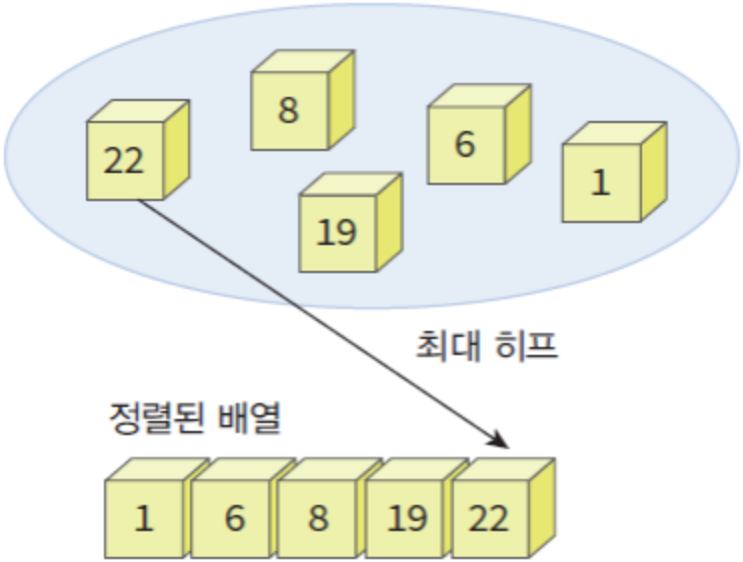


© 생능출판사 2019



히프 정렬

- 히프를 이용하면 정렬 가능





© 생능출판사 2019



히프 저려 프로그래

```
#include <stdio.h>
#include <stdlib.h>
....
// 앞의 최대 히프 코드를 여기에 추가
....
// 우선 순위 큐인 히프를 이용한 정렬
void heap_sort(element a[], int n)
{
        int i;
        HeapType* h;

        h = create();
        init(h);
        for (i = 0; i<n; i++) {
                insert_max_heap(h, a[i]);
        }
        for (i = (n - 1); i >= 0; i--) {
                a[i] = delete_max_heap(h);
        }
        free(h);
}
```



© 생능출판사 2019



히프 정렬 프로그램

```
#define SIZE 8
int main(void)
{
        element list[SIZE] = { 23, 56, 11, 9, 56, 99, 27, 34 };
        heap_sort(list, SIZE);
        for (int i = 0; i < SIZE; i++) {
                printf("%d ", list[i].key);
        }
        printf("\n");
        return 0;
}
```

```
9 11 23 27 34 56 56 99
```



© 생능출판사 2019

머신 스케줄링



JOB #1 JOB #3



JOB #2 JOB #4



© 생능출판사 2019



LPT(longest processing time first) 방법

J1	J2	J3	J4	J5	J6	J7
8	7	6	5	3	2	1

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
M1															
M2															
M3															

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
M1															
M2															
M3															



© 생능출판사 2019



LPT(longest processing time first) 방법

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
M1															
M2															
M3															

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
M1															
M2															
M3															



© 생능출판사 2019




```
#define JOBS 7
#define MACHINES 3

int main(void)
{
        int jobs[JOBS] = { 8, 7, 6, 5, 3, 2, 1 };        // 작업은 정렬되어 있다고 가정
        element m = { 0, 0 };
        HeapType* h;
        h = create();
        init(h);
```



© 생능출판사 2019



LPT

```
        // 여기서 avail 값은 기계가 사용 가능하게 되는 시간이다.
        for (int i = 0; i<MACHINES; i++) {
                m.id = i + 1;
                m.avail = 0;
                insert_min_heap(h, m);
        }
        // 최소 히프에서 기계를 꺼내서 작업을 할당하고 사용가능 시간을 증가 시킨 후에
        // 다시 최소 히프에 추가한다.
        for (int i = 0; i< JOBS; i++) {
                m = delete_min_heap(h);
                printf("JOB %d을 시간=%d부터 시간=%d까지 기계 %d번에 할당한다. \n",
                        i, m.avail, m.avail + jobs[i] - 1, m.id);
                m.avail += jobs[i];
                insert_min_heap(h, m);
        }
        return 0;
}
```



© 생능출판사 2019



JOB 0을 시간=0부터 시간=7까지 기계 1번에 할당한다.
JOB 1을 시간=0부터 시간=6까지 기계 2번에 할당한다.
JOB 2을 시간=0부터 시간=5까지 기계 3번에 할당한다.
JOB 3을 시간=6부터 시간=10까지 기계 3번에 할당한다.
JOB 4을 시간=7부터 시간=9까지 기계 2번에 할당한다.
JOB 5을 시간=8부터 시간=9까지 기계 1번에 할당한다.
JOB 6을 시간=10부터 시간=10까지 기계 2번에 할당한다.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
M1															
M2															
M3															



© 생능출판사 2019



허프만 코드

- 이진 트리는 각 글자의 빈도가 알려져 있는 메시지의 내용을 압축하는데 사용될 수 있다.
- 이런 종류의 이진트리를 허프만 코딩 트리라고 부른다.



빈도수 분석 →

A	80
B	16
C	32
D	36
E	123
F	22
G	26
H	51
I	71
...	
Z	1



© 생능출판사 2019



글자의 빈도수

- 예를 들어보자. 만약 텍스트가 e, t, n, i, s의 5개의 글자로만 이루어졌다고 가정하고 각 글자의 빈도수가 다음과 같다고 가정하자.

글자	비트 코드	빈도수	비트 수
e	00	15	30
t	01	12	24
n	10	8	16
i	110	6	18
s	111	4	12
합계			100

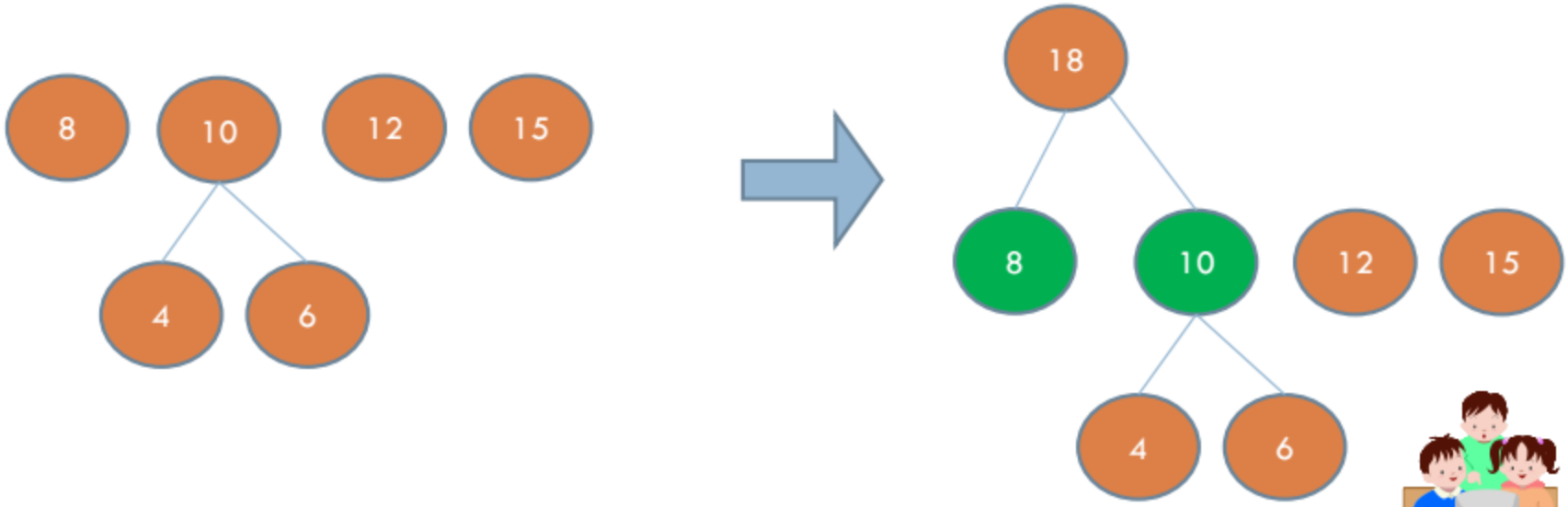


© 생능출판사 2019



허프만 코드 생성 절차



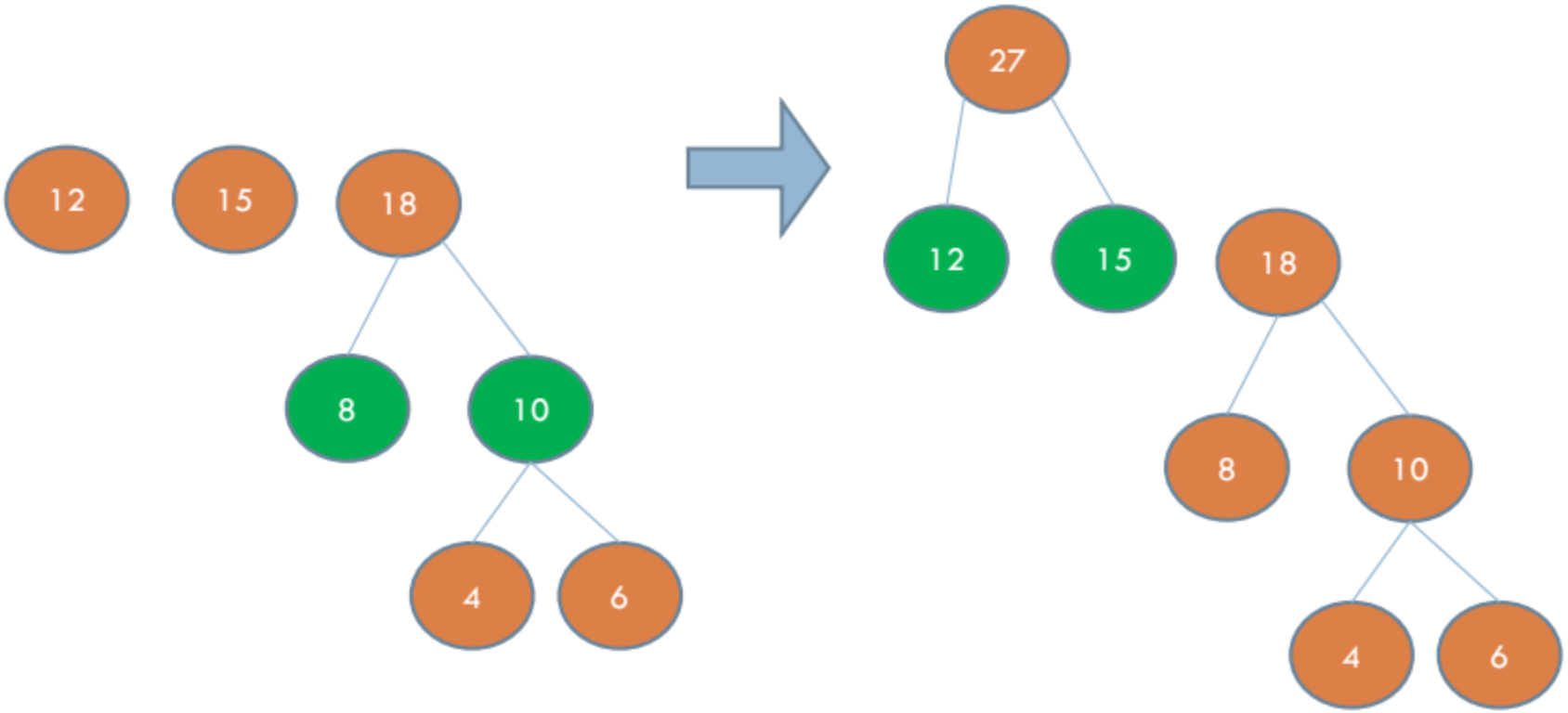




© 생능출판사 2019



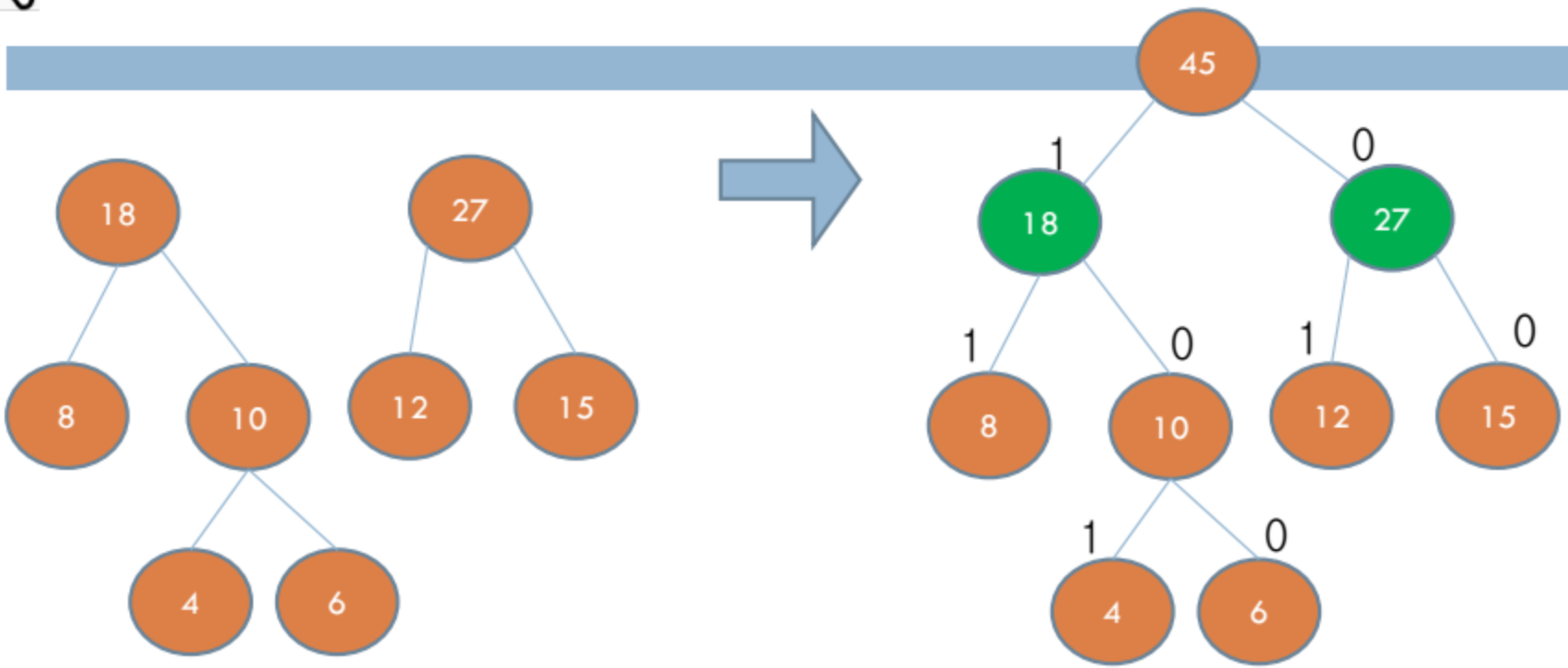
허프만 코드 생성 절차





© 생능출판사 2019

허프만 코드 생성 절차



글자	비트 코드	빈도수	비트 수
e	00	15	30
t	01	12	24
n	11	8	16
i	100	6	18
s	101	4	12
합계			100



C로 쉽게 풀(



허프만 코드 프로그램

```
#include <stdio.h>
#include <stdlib.h>
#define MAX_ELEMENT 200

typedef struct TreeNode {
        int weight;
        char ch;
        struct TreeNode *left;
        struct TreeNode *right;
} TreeNode;

typedef struct {
        TreeNode* ptree;
        char ch;
        int key;
} element;

typedef struct {
        element heap[MAX_ELEMENT];
        int heap_size;
} HeapType;
```



© 생능출판사 2019



허프만 코드 프로그램

```
// 생성 함수
HeapType* create()
{
        return (HeapType*)malloc(sizeof(HeapType));
}
// 초기화 함수
void init(HeapType* h)
{
        h->heap_size = 0;
}
// 현재 요소의 개수가 heap_size인 히프 h에 item을 삽입한다.
// 삽입 함수
void insert_min_heap(HeapType* h, element item)
{
        int i;
        i = ++(h->heap_size);

        //  트리를 거슬러 올라가면서 부모 노드와 비교하는 과정
        while ((i != 1) && (item.key < h->heap[i / 2].key)) {
                h->heap[i] = h->heap[i / 2];
                i /= 2;
        }
        h->heap[i] = item;     // 새로운 노드를 삽입
}
```



© 생능출판사 2019



허프만 코드 프로그램

```
// 삭제 함수
element delete_min_heap(HeapType* h)
{
        int parent, child;
        element item, temp;

        item = h->heap[1];
        temp = h->heap[(h->heap_size)--];
        parent = 1;
        child = 2;
        while (child <= h->heap_size) {
                // 현재 노드의 자식노드중 더 작은 자식노드를 찾는다.
                if ((child < h->heap_size) &&
                    (h->heap[child].key) > h->heap[child + 1].key)
                        child++;
                if (temp.key < h->heap[child].key) break;
                // 한 단계 아래로 이동
                h->heap[parent] = h->heap[child];
                parent = child;
                child *= 2;
        }
        h->heap[parent] = temp;
        return item;
}
```



© 생능출판사 2019



허프만 코드 프로그램

```
// 이진 트리 생성 함수
TreeNode* make_tree(TreeNode* left,
                TreeNode* right)
{
                TreeNode* node =
                                (TreeNode*)malloc(sizeof(TreeNode));
                node->left = left;
                node->right = right;
                return node;
}
// 이진 트리 제거 함수
void destroy_tree(TreeNode* root)
{
                if (root == NULL) return;
                destroy_tree(root->left);
                destroy_tree(root->right);
                free(root);
}

int is_leaf(TreeNode* root)
{
                return !(root->left) && !(root->right);
}
```



© 생능출판사 2019



```
void print_array(int codes[], int n)
{
        for (int i = 0; i < n; i++)
                printf("%d", codes[i]);
        printf("\n");
}

void print_codes(TreeNode* root, int codes[], int top)
{

        // 1을 저장하고 순환호출한다.
        if (root->left) {
                codes[top] = 1;
                print_codes(root->left, codes, top + 1);
        }

        // 0을 저장하고 순환호출한다.
        if (root->right) {
                codes[top] = 0;
                print_codes(root->right, codes, top + 1);
        }

        // 단말노드이면 코드를 출력한다.
        if (is_leaf(root)) {
                printf("%c: ", root->ch);
                print_array(codes, top);
        }
}
```



© 생능출판사 2019



허프만 코드 프로그램

```
// 허프만 코드 생성 함수
void huffman_tree(int freq[], char ch_list[], int n)
{
        int i;
        TreeNode *node, *x;
        HeapType* heap;
        element e, e1, e2;
        int codes[100];
        int top = 0;

        heap = create();
        init(heap);
        for (i = 0; i<n; i++) {
                node = make_tree(NULL, NULL);
                e.ch = node->ch = ch_list[i];
                e.key = node->weight = freq[i];
                e.ptree = node;
                insert_min_heap(heap, e);
        }
```



© 생능출판사 2019



허프만 코드 프로그램

```
        for (i = 1; i<n; i++) {
                // 최소값을 가지는 두개의 노드를 삭제
                e1 = delete_min_heap(heap);
                e2 = delete_min_heap(heap);
                // 두개의 노드를 합친다.
                x = make_tree(e1.ptree, e2.ptree);
                e.key = x->weight = e1.key + e2.key;
                e.ptree = x;
                printf("%d+%d->%d \n", e1.key, e2.key, e.key);
                insert_min_heap(heap, e);
        }
        e = delete_min_heap(heap); // 최종 트리
        print_codes(e.ptree, codes, top);
        destroy_tree(e.ptree);
        free(heap);
}
```



© 생능출판사 2019



허프만 코드 프로그램

```
int main(void)
{
        char ch_list[] = { 's', 'i', 'n', 't', 'e' };
        int freq[] = { 4, 6, 8, 12, 15 };
        huffman_tree(freq, ch_list, 5);
        return 0;
}
```

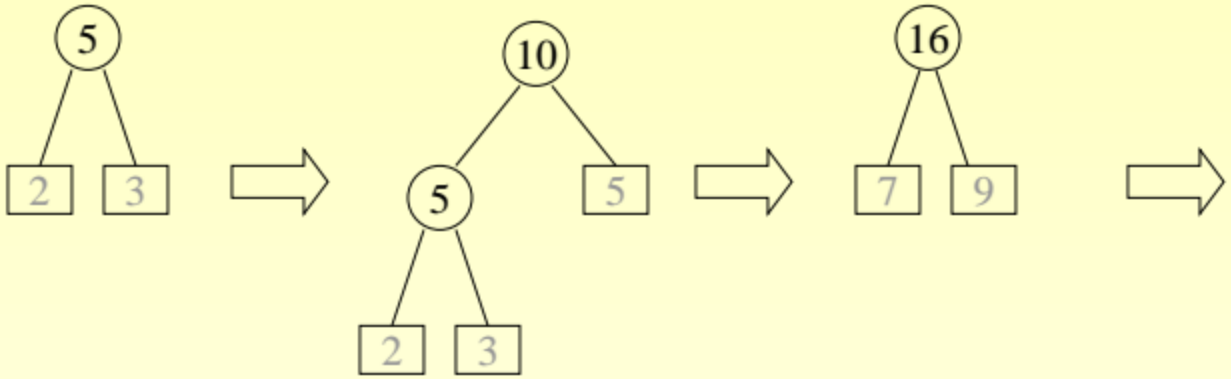
```
4+6->10
8+10->18
12+15->27
18+27->45
n: 11
s: 101
i: 100
t: 01
e: 00
```



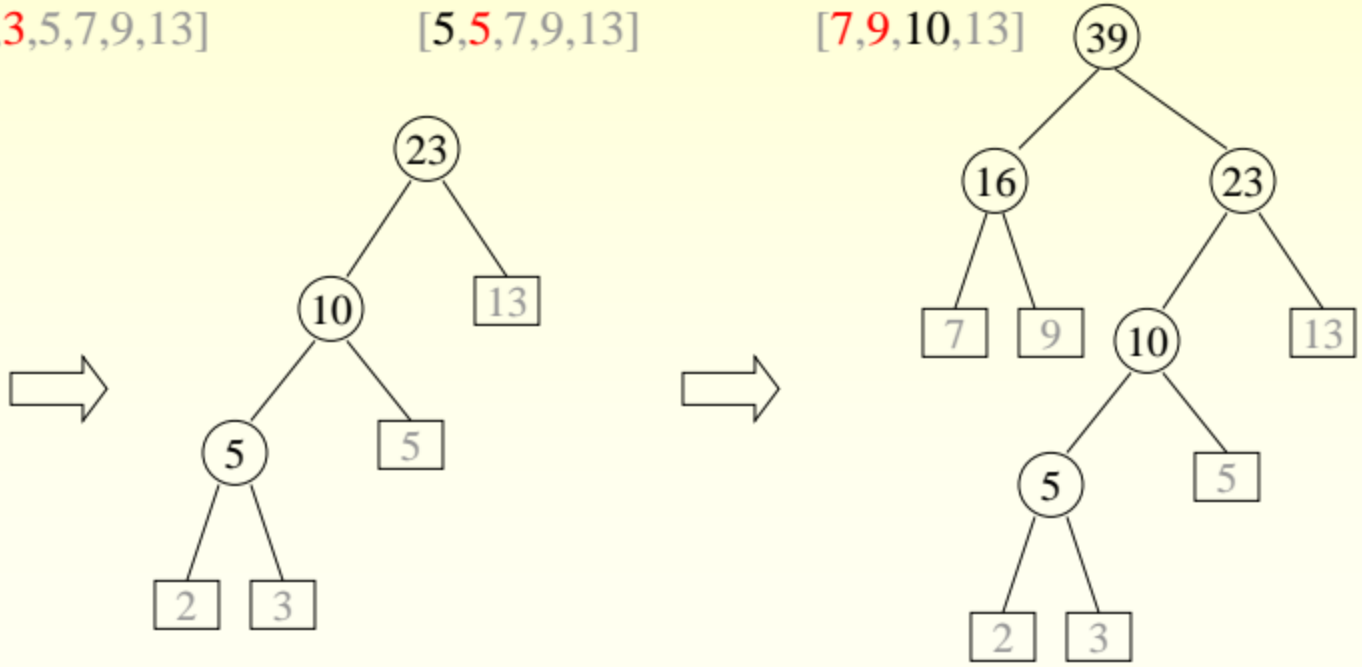
© 생능출판사 2019

허프만 트리 예 (1)

- 2, 3, 5, 7, 9, 13 의 가중치를 지닌 Huffman Tree 생성의 예



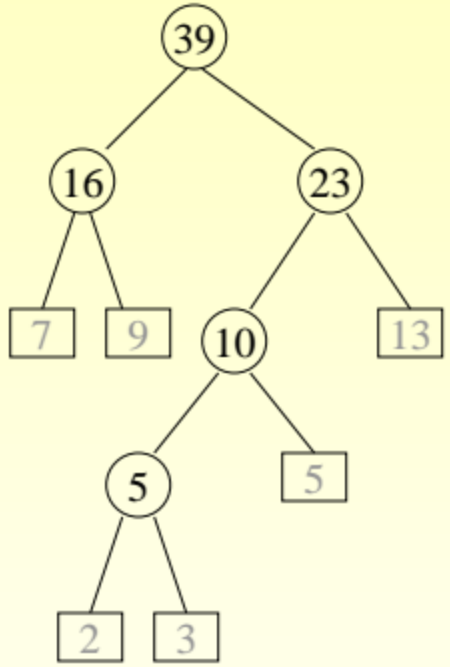
[2,3,5,7,9,13] [5,5,7,9,13] [7,9,10,13]

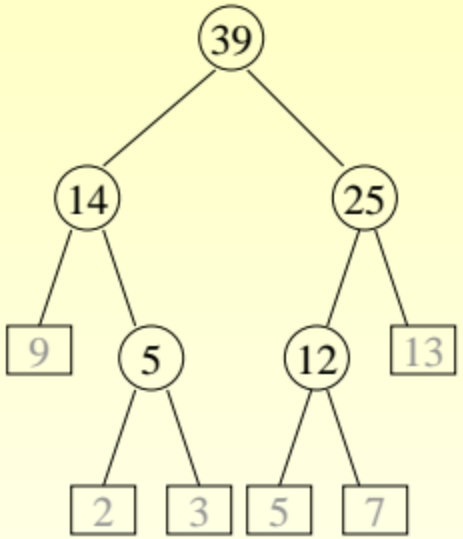


[10,13,16] [16,23] [39]

허프만 트리 예 (2)

- 2, 3, 5, 7, 9, 13 의 가중치를 지닌 Huffman Tree 생성의 예 vs. 임의로 높이를 낮춘 Tree 예





$2*4 + 3*4 + 5*3 + 7*2 + 9*2 + 13*2 = 93$

$2*3 + 3*3 + 5*3 + 7*3 + 9*2 + 13*2 = 95$